# Project Support Session

## SuperKart

# Agenda

- Project Problem Statement and Dataset Overview
- Open Q&A
- Project Evaluation Rubric
- Project Solution Approach and Submission Guidelines
- FAQs

# SuperKart: Business Context

A sales forecast predicts future sales revenue based on historical data, industry trends, and the status of the current sales pipeline. Businesses use the sales forecast to estimate weekly, monthly, quarterly, and annual sales totals. A company needs to make an accurate sales forecast as it adds value across an organization and helps the different verticals to chalk out their future course of action.

Forecasting helps an organization plan its sales operations by region and provides valuable insights to the supply chain team regarding the procurement of goods and materials. An accurate sales forecast process has many benefits, which include improved decision-making about the future and the reduction of sales pipeline and forecast risks. Moreover, it helps to reduce the time spent in planning territory coverage and establishes benchmarks that can be used to assess trends in the future.

# SuperKart: Objective

SuperKart is a retail chain operating supermarkets and food marts across various tier cities, offering a wide range of products. To optimize its inventory management and make informed decisions around regional sales strategies, SuperKart wants to accurately forecast the sales revenue of its outlets for the upcoming quarter.

To operationalize these insights at scale, the company has partnered with a data science firm, not just to build a predictive model based on historical sales data but also to develop and deploy a robust forecasting solution that can be integrated into SuperKart's decision-making systems and used across its network of stores.

# SuperKart: Data Dictionary

The data contains the different attributes of the various products and stores. The detailed data dictionary is given below.

| Data | Data Description |
|---|---|
| **Product_Id** | Unique identifier of each product, each identifier having two letters at the beginning followed by a number |
| **Product_Weight** | Weight of each product |
| **Product_Sugar_Content** | Sugar content of each product like low sugar, regular and no sugar |
| **Product_Allocated_Area** | Ratio of the allocated display area of each product to the total display area of all the products in a store |
| **Product_Type** | broad category for each product like meat, snack foods, hard drinks, dairy, canned, soft drinks, health and hygiene, baking goods, bread, breakfast, frozen foods, fruits and vegetables, household, seafood, starchy foods, others |

# SuperKart: Data Dictionary

| Data | Data Description |
|---|---|
| **Product_MRP** | Maximum retail price of each product |
| **Store_Id** | Unique identifier of each store |
| **Store_Establishment_Year** | Year in which the store was established |
| **Store_Size** | Size of the store depending on sq. feet like high, medium and low |
| **Store_Location_City_Type** | Type of city in which the store is located like Tier 1, Tier 2 and Tier 3. Tier 1 consists of cities where the standard of living is comparatively higher than its Tier 2 and Tier 3 counterparts |
| **Store_Type** | Type of store depending on the products that are being sold there like Departmental Store, Supermarket Type 1, Supermarket Type 2 and Food Mart |
| **Product_Store_Sales_Total** | Total revenue generated by the sale of that particular product in that particular store |

# SuperKart: Q&A

# SuperKart: Evaluation Rubric

| Section | Points |
|---|---|
| Data Overview and Exploratory Data Analysis | 6 |
| Data Preprocessing | 5 |
| Model Building | 7 |
| Model Performance Improvement - Hyperparameter Tuning | 7 |
| Model Performance Comparison, Final Model Selection, and Serialization | 7 |
| Deployment - Backend | 9 |
| Deployment - Frontend | 7 |
| Actionable Insights and Recommendations | 4 |
| Presentation/Notebook - Overall Quality | 8 |

# SuperKart: Evaluation Rubric

**Section 1**: Data Overview and Exploratory Data Analysis

| What to Cover? | Guiding Questions and Considerations |
| --- | --- |
| Observations on the shape of data, data types of various attributes, missing values, duplicate values and statistical summary | What technique can be used to address duplicate values (if any)?<br><br>What technique can be used to address missing values (if any)? |
| Univariate Analysis (boxplots, distribution plots for important variables) and Bivariate analysis | Univariate analysis helps understand individual variable distributions and outliers<br><br>It's important to explore relationships or patterns between different types of variables. |
| Insights based on EDA | What are the key findings based on the business problem at hand? |

# SuperKart: Evaluation Rubric

**Section 2**: Data Preprocessing

| What to Cover? | Guiding Questions and Considerations |
| --- | --- |
| Feature engineering (provide rationale if not needed) | Are new features required to be engineered for better model performance? |
| Outlier detection and treatment (provide rationale if treatment is not needed) | What methodology should be used to detect and treat outliers? |
| Prepare the data for analysis (Train and Test sets) | What would be a good ratio for splitting the data? |
| Define the preprocessing pipeline for encoding categorical features | Does the pipeline handle data leakage? |

# SuperKart: Evaluation Rubric

**Section 3**: Model Building

| What to Cover? | Guiding Questions and Considerations |
| --- | --- |
| Choose the metric of choice with proper rationale | When choosing a metric for model building, align it with the business objective |
| Build any 2 ML models as a part of a pipeline with the data preprocessing steps - Comment of the performance of the models | The ML models to be built can be any two out of Decision Tree, Bagging, Random Forest, AdaBoost, Gradient Boosting, and XGBoost. |

# SuperKart: Evaluation Rubric

**Section 4**: Model Performance Improvement - Hyperparameter Tuning

| What to Cover? | Guiding Questions and Considerations |
| --- | --- |
| Use hyperparameter tuning to tune the models wrt the metric of choice | What are the key hyperparameters that influence the model's performance?<br>What hyperparameter tuning mechanism would help balance performance and execution time? |
| Comment of the performance of the tuned models | Provide the necessary remarks for the performance of the tuned models |

# SuperKart: Evaluation Rubric

**Section 5**: Model Performance Comparison, Final Model Selection, and Serialization

| What to Cover? | Guiding Questions and Considerations |
|---|---|
| Compare the performances of all models built and choose the best model (with proper rationale) | Although the metrics of choice is key, it's important to consider other factors too for robust modeling |
| Check the performance of the best model on the test set | Does the final model generalize well? |
| Serialize the best model | The serialized artifact should contain both the model along with the preprocessing steps. |
| Load the serialized model and make predictions on the test set | Is the serialized model able to make predictions? |

# SuperKart: Evaluation Rubric

## **Section 6**: Deployment - Backend

| What to Cover? | Guiding Questions and Considerations |
| --- | --- |
| Define the Flask API, dependencies, Dockerfile | Design a lightweight REST API with clearly defined endpoints, request/response formats, and proper error handling for inference. |
| Define the dependencies | All libraries along with their corresponding version should be listed in the dependencies file |
| Define the Dockerfile | The port on which the API will run needs to be specified in the Dockerfile |

# SuperKart: Evaluation Rubric

**Section 7**: Deployment - Frontend and Inference

| What to Cover? | Guiding Questions and Considerations |
|---|---|
| Define the Streamlit app | How will your Streamlit app collect user inputs, send them to the deployed API, and display the prediction results in a user-friendly format? |
| Define the dependencies | All libraries along with their corresponding version should be listed in the dependencies file |
| Define the Dockerfile | Does your Dockerfile correctly install the dependencies, copy the application files, expose the required port, and launch the Streamlit application? |

# SuperKart: Evaluation Rubric

**Section 7**: Deployment - Frontend and Inference

| What to Cover? | Guiding Questions and Considerations |
| --- | --- |
| Push Backend and Frontend Files to the Repository | Have you organized and pushed all the necessary backend, frontend, and configuration files to your GitHub Codespace repository so the project can be reproduced without missing components? |
| Define the Forwarded URL | After running the Streamlit application in GitHub Codespaces, have you configured the forwarded port and verified that the generated public URL successfully opens your application? |

**Note: Add screenshots of the deployed Streamlit application showing a single inference along with the response and the forwarded URL**

# SuperKart: Evaluation Rubric

**Section 7**: Deployment - Frontend and Inference

| What to Cover? | Guiding Questions and Considerations |
| --- | --- |
| Online Inference – Single Prediction | Can your application accept a single set of input values, send them to the deployed model endpoint, and display the returned prediction correctly? |
| Batch Inference | Can your application accept a CSV file containing multiple records, process all the inputs through the deployed model, and present or download the batch prediction results? |

**Note: Add screenshots of the deployed Streamlit application showing a batch inference along with the response.**

# SuperKart: Evaluation Rubric

**Section 8**: Actionable Insights & Recommendations

| What to Cover? | Guiding Questions and Considerations |
|---|---|
| Write down insights from the analysis conducted and Provide actionable business recommendations | What are the key things that we learned from the data that's useful for the business?<br><br>Logical next steps are a key inclusion |

# SuperKart: Evaluation Rubric

**Section 9:** Presentation/Notebook - Overall Quality

| Low Code Considerations | Full Code Considerations |
|---|---|
| Clear structure, flow, and visual appeal - everything sits well in a story | Clear structure and flow |
| Crispness - Focus on key points | Well-commented and executed code |
| All sections of the rubric included, even if in an appendix | All sections of the rubric included, even if in an appendix |

# SuperKart: Low-code Version

For learners who aspire to be in managerial roles in the future, focusing on solution review, interpretation, recommendations, and communication with business stakeholders

Download the dataset and the Learner Notebook - Low Code (this is a template notebook)

Fill in the blanks in the notebook to complete and execute the code to solve the questions and perform all the tasks as per the grading rubric

Once the notebook is completely executed and necessary outputs obtained, a business presentation (using Microsoft PowerPoint, Google Slides, etc.) has to be created

# SuperKart: Low-code Version

The presentation should contain observations, insights, and recommendations for the business problem

The presentation template provided can be referred to as a sample

Once the presentation is complete, convert the presentation to .pdf format

**The presentation should be submitted as a PDF file (.pdf) and NOT as a .pptx file**

**Please make sure that all the sections mentioned in the grading rubric have been covered in the submission**

# SuperKart: Full-code Version

For learners who aspire to be in hands-on coding roles in the future, focusing on building solution codes from scratch

Download the dataset and the *Learner Notebook - Full Code* (this is a template notebook containing high-level steps to perform and insight-based questions)

Write necessary code to solve the questions and perform all the tasks as per the grading rubric

Clearly write down observations, insights, and recommendations for the business problem based on the analysis performed

Once the notebook is complete, download it as a **.ipynb** file and convert it to a **.html** file

# SuperKart: Full-code Version

**The notebook should be submitted as an HTML file (.html) and NOT as a notebook file (.ipynb)**

The conversion can be done via one of the following ways:

**Jupyter Notebook**: ***Download as HTML*** from the ***File*** menu
**Google Colab**: Use [free online tools]

**Please make sure that all the sections mentioned in the grading rubric have been covered in the submission**

# SuperKart: Q&A

# SuperKart: FAQ

**My Streamlit app is running, but I cannot access it in the browser. What should I check?**

Ensure  that the correct port (commonly 8501) is forwarded in GitHub Codespaces. Verify that the forwarded port is set to Public (if required) and use the generated forwarded URL to access the application.

**I generated a Personal Access Token (PAT) in Github but cannot see it again. What should I do?**

GitHub shows the token only once immediately after it is created. If you did not copy and save it, you will need to generate a new token because the old token value cannot be viewed again.

# SuperKart: FAQ

## I receive a 404 Not Found error when opening my forwarded URL. What should I do?

If your Codespace application is not accessible, first verify that the application is running without errors by checking the terminal logs. Ensure that the correct ports for both the backend and frontend services are forwarded and configured as Public. If the application has crashed during startup, identify and fix the error, restart the services, and confirm that the forwarded URLs are pointing to the correct ports before testing the application again.

## What Docker commands are used to create a network and run the churn backend container connected to it?

```
docker network create churn-app-network

docker run -d --name backend --network churn-app-network -p 7860:7860 churn-backend
```

# SuperKart: FAQ

**Why do we create a Docker network using `docker network create churn-app-network`?**

This command creates a shared bridge network so the frontend and backend containers can communicate with each other using container names (for example, the frontend can reach the backend using backend as the hostname).

**What does `docker run -d --name frontend --network churn-app-network -p 8501:8501 churn-frontend` do?**

This command starts the Streamlit frontend container in detached mode, connects it to the shared Docker network, and maps port 8501 from the container to port 8501 in GitHub Codespaces so the app can be accessed through the forwarded URL.

# SuperKart: FAQ

**Why am I getting connection errors when making predictions from the Streamlit app?**

Verify that the backend inference API is running and that the API endpoint URL configured in the Streamlit application matches the deployed backend. Also ensure there are no port or networking issues between the frontend and backend.

**My batch inference is failing after uploading a CSV file. What could be the reason?**

Check that the uploaded CSV follows the expected format, including the required column names and data types. Missing columns, extra columns, or invalid data can cause the prediction request to fail.

# Power Ahead!